BYOP (Build your own project)

Course: Programming in java | Unit Coverage: 1 – 5 Max Marks: 100

Student Name	Registration Number
Shorya Pratap SIngh	24BCY10298
Slot	**Date of Submission**
A21+A22	31/**03/2026**

I INTRODUCTION

In today's world, where everything is becoming digital, managing data efficiently has become very important, especially in the field of education. Schools, colleges, and universities deal with a large amount of student information every day. This includes details like student names, IDs, academic records, attendance, and more. Handling all this information manually is not only slow but also increases the chances of mistakes.

Earlier, most institutions used paper-based methods such as registers and files to store student data. While this system worked in the past, it has many drawbacks. It takes up a lot of physical space, makes it difficult to find specific records quickly, and increases the chances of duplicate or incorrect data. Also, paper records can easily get damaged, lost, or accessed by unauthorized people.

With the growth of technology, it has become necessary to move from manual systems to digital solutions. A Student Management System is one such solution that helps in managing student data in an organized and efficient way. It allows users to easily store, access, update, and delete student records, which saves time and reduces manual effort.

In this project, a simple Student Management System is developed using the Java programming language. Java is used because it is easy to learn, works on different platforms, and supports object-oriented programming. The system is designed as a console-based application where users can perform basic operations like adding new students, viewing existing records, and deleting records when needed. A menu-driven approach is used so that even beginners can use the system easily.

An important part of this project is the use of object-oriented programming concepts. A separate Student class is created to store student details, while the main program handles user input and operations. The system also uses ArrayList, which helps in storing data dynamically without any fixed limit.

The main goal of this project is to show how basic programming concepts can be used to solve real-life problems. It also acts as a starting point for developing more advanced systems in the future, such as applications with graphical interfaces, databases, or web-based features.

Working on this project also helps in improving problem-solving skills, logical thinking, and understanding of software development. It gives practical experience in using tools like Java, coding environments, and version control platforms.

In conclusion, the Student Management System is a simple yet useful application that makes managing student data easier and more efficient. It reduces manual work, minimizes errors, and improves data handling. This project not only meets academic requirements but also builds a strong base for creating more advanced applications in the future.

•

II LITERATURE REVIEW

Before developing the Student Management System, it is important to understand how similar systems have been used and studied in the past. Many researchers and developers have worked on systems that help in managing student data digitally, and these systems have shown how technology can improve efficiency in educational institutions.

In earlier times, most institutions relied on manual methods such as registers and paper files to maintain student records. Studies have shown that these methods are slow, difficult to manage, and highly prone to errors. As the number of students increases, maintaining records manually becomes even more complicated and inefficient.

With the introduction of computer-based systems, many solutions were developed to handle student information more effectively. Basic software applications were created to store student data in digital form, making it easier to search, update, and manage records. These systems reduced paperwork and helped in saving time and effort.

In recent years, more advanced Student Management Systems have been developed using modern technologies such as databases, web applications, and cloud platforms. These systems allow multiple users to access data from different locations and provide features like attendance tracking, result management, and report generation. However, such systems can be complex and require more technical knowledge and resources.

For beginners and small-scale applications, simple console-based systems are still very useful. They help in understanding the basic concepts of programming and data management without adding unnecessary complexity. Many academic projects focus on developing such systems using programming languages like Java, as it provides a strong foundation in object-oriented programming.

This project is inspired by these existing systems but focuses on creating a simple and easy-to-understand version. It uses basic features such as adding, viewing, and deleting student records. The use of ArrayList for data storage makes the system flexible and suitable for small-scale use.

Overall, the literature suggests that digital systems are far more efficient than manual methods for managing student data. Even though advanced systems exist, starting with a simple implementation helps in building a clear understanding of how such systems work. This project follows the same approach and serves as a stepping stone toward more advanced applications in the future.

III TOOLS AND TECHNOLOGY USED

The development of the Student Management System involves the use of several tools and technologies that help in designing, coding, testing, and managing the project effectively. Each tool plays an important role in ensuring that the system is developed smoothly and works efficiently.

1. Java Programming Language

Java is the main programming language used to develop this project. It is a high-level, object-oriented programming language that is widely used for building applications. One of the main advantages of Java is that it is platform-independent, which means the program can run on different operating systems without modification.

Java also supports important concepts such as classes, objects, inheritance, and encapsulation, which are used in this project to create a structured and modular system. Its simple syntax and strong community support make it a good choice for beginners as well as professionals.

2. Java Development Kit (JDK)

The Java Development Kit (JDK) is required to compile and run Java programs. It includes important tools such as the Java compiler (javac) and the Java Runtime Environment (JRE). The compiler converts the Java code into bytecode, which can then be executed on any system that has Java installed.

Without JDK, it is not possible to develop or run Java applications, so it is an essential component of this project.

3. Integrated Development Environment (IDE)

An Integrated Development Environment (IDE) is used to write, edit, and run the code efficiently. In this project, tools like VS Code or IntelliJ IDEA can be used.

These IDEs provide useful features such as:

- Syntax highlighting (to make code readable)
- Error detection and debugging
- Auto-completion of code
- Easy execution of programs

Using an IDE makes the development process faster and more organized compared to writing code in a simple text editor.

4. Java Collections Framework (ArrayList)

The project uses ArrayList, which is a part of the Java Collections Framework. It is used to store student records dynamically.

Unlike arrays, ArrayList does not have a fixed size, which means it can grow or shrink as needed. This makes it very useful for storing data where the number of records is not known in advance.

Each student is stored as an object in the ArrayList, making it easy to add, remove, and access records.

5. Git (Version Control System)

Git is a version control system used to track changes in the project. It helps developers manage different versions of their code and keeps a history of all changes made.

Using Git, it becomes easier to:

- Save different versions of the project
- Undo mistakes if needed
- Collaborate with others

Even though this is a small project, using Git introduces good development practices.

6. GitHub (Project Hosting Platform)

GitHub is an online platform used to store and share the project repository. It allows developers to upload their code and access it from anywhere.

In this project, GitHub is used to:

- Upload project files such as Java code, README, and report
- Share the project link for submission
- Maintain a structured repository

GitHub also makes the project look more professional and organized.

7. Operating System

The project can be developed and executed on any operating system such as Windows, Linux, or macOS. In most cases, Windows is used due to its user-friendly interface.

The operating system provides the environment where all tools such as Java, IDE, and Git are installed and used.

8. Microsoft Word (Documentation Tool)

Microsoft Word is used to create the project report. It helps in organizing the content in a structured format with headings, tables, images, and proper alignment.

Screenshots of the code and output are also inserted into the document to make the report more informative and visually clear.

Overall, the combination of these tools and technologies helps in building a simple yet effective Student Management System. Each tool contributes to different stages of development, making the project easy to implement, manage, and present.

IV Methodology

The development of the Student Management System follows a simple and structured approach to ensure that the application is easy to understand, efficient, and functional. The methodology focuses on breaking the project into smaller steps so that each part can be developed and tested properly.

1. Requirement Analysis

The first step in the development process was to understand the problem and identify the requirements of the system. The main goal was to create a system that can manage student records efficiently. Based on this, the key functionalities were decided, such as adding a student, viewing student details, and deleting a student record.

2. System Design

After identifying the requirements, the next step was to design the structure of the system. The system is designed using object-oriented programming concepts. A separate class named *Student* is created to store the details of each student, such as ID, name, and age. The main class handles the program execution and user interaction.

A menu-driven design is used so that users can easily select the desired operation. This makes the system simple and user-friendly.

3. Implementation

In this step, the actual coding of the system is done using the Java programming language. The program is written in a way that it continuously runs until the user chooses to exit.

The main features implemented include:

- Adding a new student record
- Displaying all student records
- Deleting a student record

Conditional statements and loops are used to control the flow of the program. The system takes input from the user and performs the required operation accordingly.

4. Data Storage

The student data is stored using an ArrayList, which is a part of the Java Collections Framework. ArrayList is chosen because it allows dynamic storage, meaning there is no fixed limit on the number of student records.

Each student object is stored in the ArrayList, making it easy to add, remove, and access records when needed.

5. Testing

After implementation, the system is tested with different inputs to ensure that all features work correctly. Various test cases are used, such as adding multiple students, deleting a student, and viewing the list of students.

Testing helps in identifying and fixing errors, ensuring that the system runs smoothly without any issues.

6. Execution

Once testing is complete, the program is executed. The system displays a menu on the console, allowing the user to choose different operations. Based on user input, the system performs the selected task and displays the result.

7. Documentation

Finally, all the steps, code, outputs, and explanations are documented in the project report. Screenshots of the code and output are included to provide a clear understanding of how the system works.

The overall development process can be summarized in the following table:

Step No.	Phase	Description
1	Requirement Analysis	Identify system needs
2	System Design	Create structure
3	Implementation	Write Java code
4	Data Storage	Use ArrayList
5	Testing	Check system
6	Execution	Run program

V IMPLEMENTATION

The implementation phase focuses on the actual development of the Student Management System using Java. In this phase, the design of the system is converted into a working program. The system is developed using object-oriented programming concepts and follows a simple menu-driven approach.

The implementation is divided into different parts such as class creation, data storage, and user interaction.

7.1 Student Class Implementation

The Student class is used to represent individual student records. It contains attributes such as student ID, name, and age. A constructor is used to initialize these values when a new object is created.

```java
class Student {
    int id;
    String name;
    int age;

    Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }
}
```

Figure 1: Student Class Implementation

7.2 Main Class and Data Storage

The main class controls the execution of the program. It uses an ArrayList to store student objects dynamically.

```java
public class Main {

    static ArrayList<Student> students = new ArrayList<>();


    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

Figure 2: Data Storage using ArrayList

7.3 Menu-Driven Logic

The system uses a menu-driven interface where users can choose different operations. A loop is used to keep the program running until the user exits.

```java
while (true) {
    System.out.println(x: "\n===== Student Management System =====");
    System.out.println(x: "1. Add Student");
    System.out.println(x: "2. View Students");
    System.out.println(x: "3. Delete Student");
    System.out.println(x: "4. Exit");
    System.out.print(s: "Enter your choice: ");
```

Figure 3: Menu-Driven Interface

7.4 Add Student Functionality

This feature allows the user to add a new student by entering details such as ID, name, and age.

```java
public static void addStudent(Scanner sc) {
    System.out.print(s: "Enter Student ID: ");
    int id = sc.nextInt();
    sc.nextLine(); // clear buffer

    System.out.print(s: "Enter Student Name: ");
    String name = sc.nextLine();

    System.out.print(s: "Enter Student Age: ");
    int age = sc.nextInt();

    students.add(new Student(id, name, age));
    System.out.println(x: "✅ Student Added Successfully!");
}
```

Figure 4: Adding Student Record

7.5 View Student Functionality

This feature displays all stored student records.

```java
public static void viewStudents() {
    if (students.isEmpty()) {
        System.out.println(x: "No records found.");
        return;
    }

    System.out.println(x: "\n--- Student List ---");
    for (Student s : students) {
        System.out.println("ID: " + s.id + ", Name: " + s.name + ", Age: " + s.age);
    }
}
```

Figure 5: Viewing Student Records

7.6 Delete Student Functionality

This feature removes a student record based on the given ID.

```java
public static void deleteStudent(Scanner sc) {
    System.out.print(s: "Enter Student ID to delete: ");
    int delId = sc.nextInt();

    boolean removed = students.removeIf(s -> s.id == delId);

    if (removed) {
        System.out.println(x: "🗑 Student Deleted Successfully!");
    } else {
        System.out.println(x: "Student not found.");
    }
```

Figure 6: Deleting Student Record

Overall, the implementation of the system is simple and effective. Each functionality is implemented using basic Java concepts, making the system easy to understand and use.

Module	Description
Student Class	Stores student data
Add Module	Adds new record
View Module	Displays records
Delete Module	Removes record

VI OUTPUT

The output of the Student Management System is displayed on the console screen. The program follows a menu-driven approach, allowing the user to select different operations such as adding, viewing, and deleting student records.

When the program starts, a menu is displayed with multiple options. The user can enter their choice, and the system performs the selected operation accordingly.

8.1 Program Menu

When the program is executed, the following menu is displayed:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\student-management-system> cd "d:\student-management-system\" ;
if ($?) { javac Main.java } ; if ($?) { java Main }

===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice:
```

The menu provides options for adding a student, viewing student records, deleting a student, and exiting the program.

8.2 Adding a Student

When the user selects the "Add Student" option, the system asks for student details such as ID, name, and age. After entering the details, the student record is successfully added.

```
===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice: 1
Enter Student ID: 1
Enter Student Name: Shorya
Enter Student Age: 18
â?? Student Added Successfully!

===== Student Management System =====
```

Figure 8: Adding a Student Record

8.3 Viewing Students

When the user selects the "View Students" option, all stored student records are displayed on the screen in a structured format.

```
===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice: 2

--- Student List ---
ID: 1, Name: Shorya, Age: 18

===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice:
```

Figure 9: Viewing Student Records

8.4 Deleting a Student

When the user selects the "Delete Student" option, the system asks for the student ID. If the ID exists, the record is deleted successfully.

```
===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice: 3
Enter Student ID to delete: 1
???? Student Deleted Successfully!

===== Student Management System =====
1. Add Student
2. View Students
3. Delete Student
4. Exit
Enter your choice:
```

Figure 10: Deleting a Student Record

The output of the system is simple, clear, and easy to understand. It ensures smooth interaction between the user and the system. All operations are performed quickly, and the results are displayed immediately on the console.

Overall, the output confirms that the system is functioning correctly and meets the required objectives of managing student data efficiently.

VII DISCUSSION

The Student Management System developed in this project provides a simple and effective way to manage student records using basic programming concepts. The system was designed to be easy to use and understand, especially for beginners, while still solving a real-world problem related to data management in educational institutions.

During the development of this project, it was observed that using a digital system significantly improves the efficiency of managing student data compared to traditional manual methods. Tasks such as adding, viewing, and deleting records can be performed quickly and accurately without the risk of data loss or duplication. The menu-driven approach makes the system user-friendly and ensures smooth interaction between the user and the program.

The use of object-oriented programming played an important role in organizing the system. By creating a separate Student class, the data is structured properly, and the code becomes more readable and maintainable. This approach also makes it easier to extend the system in the future by adding more features.

The choice of ArrayList for data storage proved to be effective for this type of application. It allows dynamic storage of student records, which means there is no limitation on the number of entries. Operations like adding and deleting records are handled efficiently, making the system flexible and scalable to some extent.

However, the system also has some limitations. Since it is a console-based application, it does not provide a graphical interface, which may not be very attractive or convenient for all users. Additionally, the data is stored temporarily in memory, meaning that all records are lost once the program is closed. There is also no search or update feature, which could make the system more useful in real-world scenarios.

Despite these limitations, the project successfully achieves its main objective of demonstrating how programming can be used to solve practical problems. It provides a strong foundation for further development and improvement. Future enhancements can include adding a graphical user interface (GUI), integrating a database for permanent data storage, and implementing additional features like search, update, and user authentication.

Overall, this project highlights the importance of using technology to improve efficiency and accuracy in data management. It also shows how even a simple system can make a significant difference when compared to traditional methods.

Aspect	Advantages	Limitations
Data Management	Stores student data in an organized way	Data is not stored permanently (lost after program ends)
Speed	Faster than manual record keeping	Performance may slow if data becomes very large
Accuracy	Reduces human errors during data entry	No validation for incorrect input
User Interface	Simple and easy to use (menu-driven)	No graphical user interface (GUI)
Storage	Uses dynamic storage (ArrayList)	No database integration
Accessibility	Easy to access and manage data	Cannot be accessed remotely

Aspect	Advantages	Limitations
Security	Better than paper-based systems	No authentication or login system
Flexibility	Easy to modify and extend	Limited features in current version
Cost	Low cost, no extra tools required	Not suitable for large-scale institutions
Maintenance	Easy to maintain and update code	Requires programming knowledge for changes
Efficiency	Saves time and effort	Limited automation features
Scalability	Can be improved in future	Not scalable in current form
Reliability	Works consistently without manual errors	Data loss risk due to no backup
Learning Value	Helps understand Java and OOP concepts	Not a complete real-world solution

VII CONCLUSION

The Student Management System developed in this project successfully demonstrates how basic programming concepts can be used to solve real-world problems in an efficient and structured way. The system provides a simple and user-friendly interface for managing student records, including adding, viewing, and deleting data.

Through this project, the limitations of traditional manual systems have been clearly addressed. The digital approach reduces the chances of errors, saves time, and makes data handling much easier and more organized. The use of Java programming and object-oriented concepts has helped in creating a modular and maintainable system.

One of the key achievements of this project is the implementation of a menu-driven system that allows users to interact with the application easily. The use of ArrayList for dynamic data storage ensures flexibility and efficient data management. The project also helped in understanding important concepts such as classes, objects, loops, and conditional statements.

In addition to technical knowledge, this project has improved problem-solving skills, logical thinking, and the ability to design simple software solutions. It also provided practical experience in using development tools and version control systems.

Although the system is basic, it has the potential to be enhanced further. Future improvements may include adding a graphical user interface (GUI), integrating a database for permanent storage, implementing a search feature, and adding user authentication for security.

In conclusion, the Student Management System is a useful and effective application that fulfills its intended purpose. It not only meets the academic requirements but also serves as a foundation for developing more advanced and scalable software systems in the future.